ML Types

1. **Supervised Learning:** Learn from data with known correct answers
    a. Tasks:
        i. Regression: Predict a number
        ii. Classification: Predict a category
        iii. Ranking: Predict order
        iv. Sequence Prediction: Predict a sequence
    b. Variations:
        i. Semi-Supervised Learning: Use a small amount of data with correct labels and a large amount of data without labels to train the model
            1. Train on labeled data
            2. Predict unlabeled data
            3. Select high-confidence predictions
            4. Add to training data
            5. Retrain and repeat
        ii. Self-Supervised Learning: The model creates its own labels from the data
2. **Unsupervised Learning:** Learn from data without known answers to find patterns or groups in the data
    a. Tasks:
        i. Clustering: Group similar data points without known labels
        ii. Association Rule Mining: Discover relationships between items in large datasets
        iii. Dimensionality Reduction: Reduce the number of features while keeping important information
3. **Reinforcement Learning:** Learn by trying actions, receiving rewards or penalties and improving decisions over time
    a. Tasks:
        i. Decision Making: Choose the best action
        ii. Control: Control a system

ML Algorithms

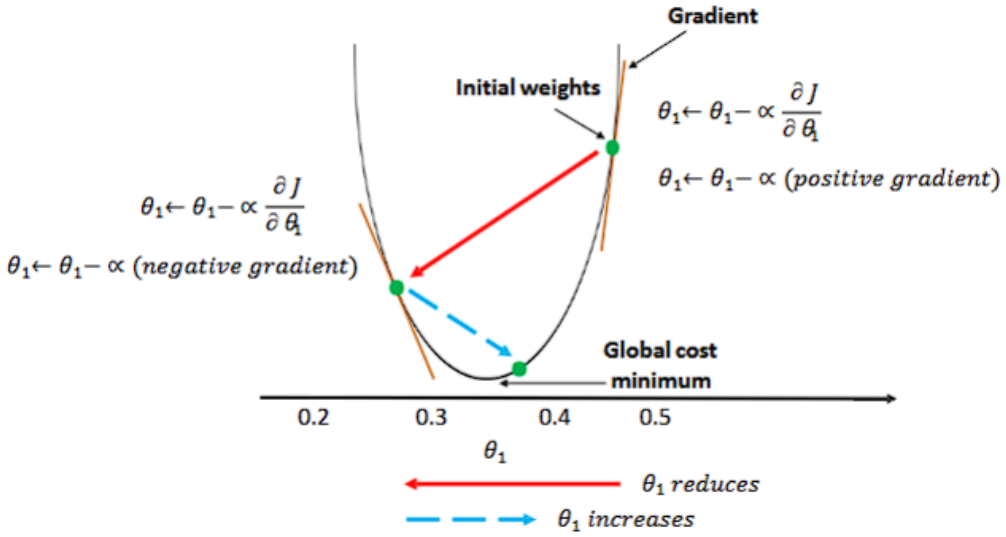
1. Linear Regression

Predict a continuous value using a linear relationship between variables

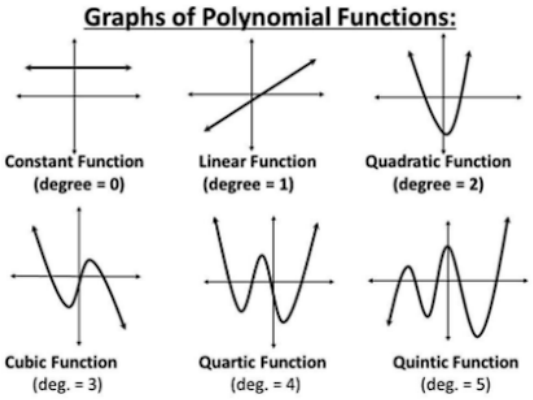
- Steps:
  - **Model Assumption:** $y = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$
  - **Prediction:** $\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$
  - **Loss Function:** $L = (y - \hat{y})^2$
  - **Training Goal:** Find parameters ($w_1, w_2, \ldots, w_n, b$) that minimize the loss function
  - **Gradient Descent:** This iterative method produces approximate values of the parameters
    - Start with random parameters
    - Compute the loss
    - Adjust the parameters slightly to reduce the loss
    - Repeat many times until the loss becomes small

      $w = w - \alpha\frac{\delta L}{\delta W}$

      where α is the learning rate and L is the loss function
- Types of Linear Regression:
  - **Simple Linear Regression:** Refers to a method used when dealing with a single independent variable (X)
  - **Multiple Linear Regression:** Refers to a method used when dealing with multiple independent variables (X1, X2, X3, …)
- Finding parameters in Linear Regression:
  - The method used to determine the parameters in a linear regression equation is **OLS (Ordinary Least Squares)**
    - Least Square Cost Function: $J(w, b) = \sum_{n=1}^{N} (y^n - (wx^n + b)^2$
    - The Least Squares cost function is Convex (single global minimum)
    - To reach the minimum: $\frac{\delta J}{\delta w} = 0$ and $\frac{\delta J}{\delta b} = 0$
  - **Normal Equation (direct):** Uses algebra to solve for the optimal parameters
    - This works well for small datasets but becomes computationally expensive for large datasets due to matrix inversion
    - Equation: $\theta = (X^TX)^{-1}X^Ty$
  - **Gradient Descent (iterative):** Gradually adjusts the parameters until optimal
    - This is efficient for large datasets, it requires choosing a learning rate, which control the step size for updates
    - It starts with initial parameter values and gradually updates them in the direction of the steepest decrease, guided by the derivative (gradient). The process repeats until it reaches the global minimum



- Polynomial Regression:
  - This is an extension of linear regression that models the relationship between the independent variable X and the dependent variable Y using polynomial terms. Instead of fitting a straight line, it fits a curve by adding higher-degree terms of X
  - **Formula:** $h_\theta(x) = \theta_0 x^0 + \theta_1 x^1 + \theta_2 x^2 + \ \ldots \ + \theta_M x^M$
  - This is considered a special case of MLR (Multiple Linear Regression)



- Overfitting:
  - Occurs when a model learns the noise and details of the training data too well, capturing unnecessary patterns that do not generalize to new data. The model performs exceptionally well on training data but poorly on test data
  - **Causes:**
    - High model complexity: Too many parameters capture noise
    - Small dataset: Not enough data to generalise
    - Noisy/irrelevant features: Model learns unimportant patterns
    - Too many epochs: Model memorises training data

- Underfitting:
  - Occurs when a model is too simple to capture the underlying pattern in the data. The model performs poorly on both training and test data, meaning it has not learned enough from the data
  - **Causes:**
    - Simple model: Lacks capacity for complex patterns
    - Few features: Important information missing
    - Insufficient training: Not enough epochs
    - Wrong model choice: Basic model for complex data
- Regularisation: Reduces model complexity by adding a penalty to large weights
  - **L1 Regularization (Lasso):**

$$J(\theta) = \frac{1}{2N}\left[\sum_{n=1}^{N}\left(h\ (x^{(n)}) - y^{(n)}\right)^2 + \boxed{\lambda \sum_{j=1}^{M} |\ \theta_j\ |}\right]$$

L1 regularization term

    - This uses the absolute values of weights as a penalty term
    - A regression model that applies L1 regularisation is called Lasso Regression
    - Unique property: Shrinking some weights to 0, effectively removing less important features from the model
      → Can be used for feature selection, identifying the most relevant features in a dataset
  - **L2 Regularization (Ridge):**

$$J(\theta) = \frac{1}{2N}\left[\sum_{n=1}^{N}\left(h\ (x^{(n)}) - y^{(n)}\right)^2 + \boxed{\lambda \sum_{j=1}^{M} \theta_j^{\,2}}\right]$$

regularization term

• Goal: minimise J(θ)

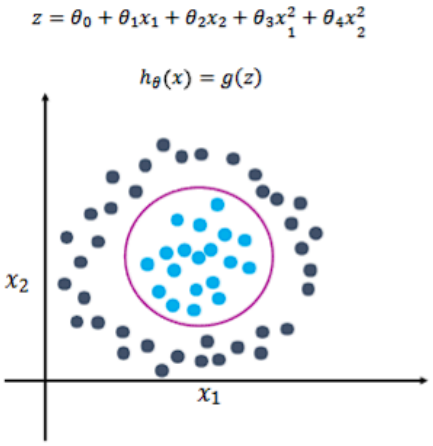
Regularisation parameter (weight decay)

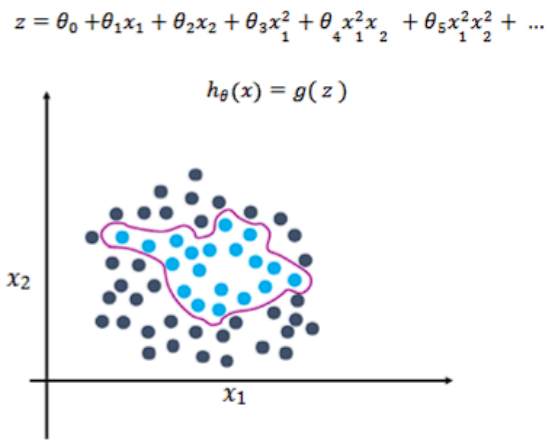
    - To minimize $J(\theta)$, the learned model will try to shrink the regularisation term by reducing the θ_i towards 0
    - The values of θ_i become smaller, leading to a simpler model
    - When λ is too large, the learned model will force the θ_i to shrink to a larger extent towards 0. The hypothesis will become almost linear
    - A very large λ value may cause underfitting in the training set
    - A very small λ value may make the regularisation term has little or no effect n regularising θ_i

2. Logistic Regression

Predict the probability of a class for classification tasks

- **Model Assumption:** The sigmoid function converts the result into a probability between 0 and 1, so we can use logistic regression for binary classification
  $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$
  $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$
- **Prediction:** For each training data point, the model predict:
  $\hat{y} = P(y = 1|x)$
  This represents the probability that the input belongs to class 1
- **Loss function:** Cross-Entropy Loss
  $L = -\sum_{i=1}^{n}[y_i log(\hat{y}_i) + (1 - y_i) log(1 - \hat{y}_i)]$
- **Optimisation:** The parameters are usually learned using gradient descent
- Logistic Regression is used for Classification problems, it predicts the probability of a Categorical dependent variable y, which could be represented by 0 or 1
  → It predicts $P(y = 1|x; \theta)$ as a function of x, given parameters of θ
- **Logistic Regression equation**: $h(x) = \frac{1}{1+e^{-(\theta_0+\theta_1 x)}}$
- **Decision boundaries:** Separates the data points into decision regions, which are actually the classes in which they belong. After training a ML model using a dataset, it is often necessary to visualize the classification of the data-points in Feature Space





- Linear versus Logistic Regression:
  - Linear:
    - Data is modelled by a **Linear function**
    - Necessary that a linear relationship be established among dependent and independent variable
    - The dependent target output y is **continuous**
  - Logistic:
    - Data is modelled by **S-shaped logistic function**
    - Not Necessary that a linear relationship be established among dependent and independent variable
    - The dependent target output, y is **discrete**
- **Cost function:**
  - Using linear regression cost function $J(\theta) = \frac{1}{2N}\sigma_{n=1}^{N}(h_\theta x^n - y^n)^2$ for logistic regression may result in a non-convex function due to the $h_\theta x^n$ is a non-linear function
  - Binary Cross-Entropy:

    $J(\theta) = -\frac{1}{N}\sum_{i=1}^{N}[y^i log(x^i h_\theta) + (1 - y^i) log(1 - log(x^i h_\theta))]$
    - $J(\theta)$ represents the cost function
    - N is the number of training examples
    - y^i is the actual label (0 or 1) of the ith training example
    - $x^i h_\theta$ is the predicted probability that the ith training example belongs to class 1, given by the sigmoid function applied to the input x^i and the model parameters θ
- Multiclass Classification:
  - **One-vs-All binary Logistic Regression**: Train a Logistic Regression classifier $h^c(x)$for each class c to predict $P(y = c|x; \theta)$
  - **Multinomial Logistic Regression:** This is an extension of binary Logistic Regression that allows for more than 2 categories in the dependent variable
    - $z = [z1, z2, z3]$

      Where $z_c = \theta_0^c + \theta_1^c x_1 + \theta_2^c x_2 = \theta^{cT} x$

      $h_\theta(x) = softmax(z)$
    - The **Softmax function** is used to convert raw scores into probabilities while ensuring that the probabilities sum up to 1 across all categories

$$softmax(z_c) = \frac{e^{z_c}}{\sum_{k=1}^{K} e^{z_k}}$$

        - Cost function: Because Multinomial Logistic Regression uses the Softmax, its cost function will be **Cross-Entropy:**

          $$J(\Theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} log(\widehat{p}_{ik})$$

          Where:

          Θ represents all the parameter vectors
          N is the number of observations
          y_{ik} is an indicator variable that equals 1 if i belongs to k
          $\widehat{p}_{ik}$ is the predicted probability that i belongs to category k

- Gradient Descent variation:
    - **Batch Gradient Descent:**
        - This calculates the error for each example in the training dataset but only updates the model after all training examples have been evaluated
        - 1 training **epoch** means 1 cycle through the entire training dataset
        - Pros:
            - Fewer updates to the model → computationally efficient
            - Produces a stable error gradient and a **stable convergence**
        - Cons:
            - A stable error gradient can sometimes result in **premature convergence** of the model to a less optimal set of parameters
            - **Resource exhausted** as it requires the entire training set resides in memory
            - Model updates, and and therefore the speed of training can become **very slow for large datasets**
    - **Mini-batch Gradient Descent:**
        - This splits the training dataset into mini batches and calculates the error and updates the model for each mini batch
        - Pros:
            - Update frequency that is higher than batch gradient descent allows for a more **robust convergence**, **avoiding local minima**
            - **Resource efficiency**
            - Mini-batch size < total training set
              → Adding noise to the learning process
              → **Improve the generalization error**
        - Cons:
            - **Additional hyperparameter** which is the mini-batch size for the learning algorithm

      → *Mini-batch GD which uses a combination of SGD and Batch GD is often a preferred method to be used*

    - **Stochastic Gradient Descent (SGD):**
        - This calculates the error and updates the model for each example in the training dataset
        - Pros:
            - Adding more noise to the learning process than mini-batch → **Improve generalization error**
            - Faster updates can result in faster learning on some problems
            - **Resource efficiency**
        - Cons:
            - Frequent updates is more **computationally expensive** than the other gradient descent configurations
            - The noisy learning process causes the model **harder to converge**
- Logistic Regression in Python
    - **Import libraries:**

```
Import numpy as np
Import pandas as pd
From sklearn import linear_model
```

    - **Split training and testing data:**

```
From sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test=train_test_split(x,y,test_size=0.2,
random_state=0)
```

        - test_size: The proportion of the dataset that is allocated for testing
        - random_state: controls the randomness of the data splitting process. When random_state is an integer, it acts as a seed, ensuring the data splitting process is reproducible
    - **Train the Logistic Regression classifier:**

```
clf=linear_model.LogisticRegression(C=1e40,solver='newton-cg')
fitted_model=clf.fit(X_train,Y_train)
```

        - C represents the inverse of regularization strength, default 1.0. Regularization is a technique used to prevent overfitting by penalizing large coefficients
        - solver: Specifies the optimization algorithm to use in the logistic regression model
    - **Evaluate model:**

```
from sklearn.metrics import accuracy_score,
precision_score,recall_score,f1_score
```

        - Predict on the test data:

```
y_pred=clf.predict(X_test)
```

        - Calculate evaluation scores:

```
accuracy=accuracy_score(y_test,y_pred)
precision=precision_score(y_test,y_pred)
recall=recall_score(y_test,y_pred)
f1=f1_score(y_test,y_pred)
```

        - Print scores:
          ```
          print(f"Accuracy:{accuracy}")
          ...
          ```
    - **Make predictions:**
      ```
      data=pd.DataFrame([[1, 2, 48],[2, 0, 45]],
          columns=['feature1', 'feature2', 'feature3'])
      ```
        - Make predictions on new data:
          ```
          predictions=clf.predict(data)
          ```
        - Print the predicted class labels:
          ```
          print(predictions)
          ```

→ The prediction is an array containing all the predicted class labels for the new data

3. Decision Tree

Make predictions using a tree of decision rules

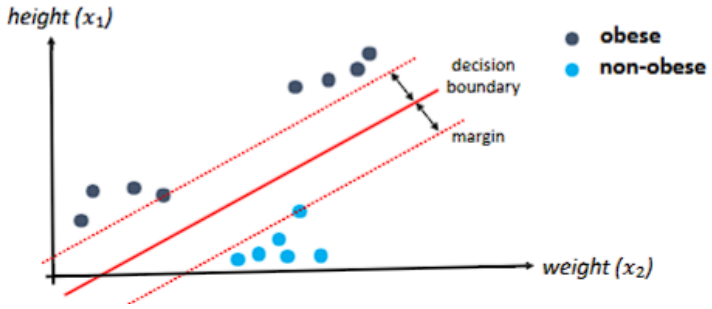
1. Start with all training data
2. Find the best feature and rule to split the data into smaller groups
3. Repeat the process for each group
4. Stop when the groups are pure or small enough
5. Assign a prediction to each final group (leaf node)

4. SVM (Support Vector Machine)

SVM tries to find a decision boundary that separates the classes with the largest margin. The margin is the distance between the boundary and the closet data points. These closet points are called support vectors. SVM is also called a large-margin classifier

- **Separating hyperplane:**
  $w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = 0$
- **Prediction:** For an input x, the model computes:
  $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$
  If z > 0 predict class 1, otherwise predict class -1
- **Hinge Loss:**
  $L = max(0, 1 - y(w_1x_1 + w_2x_2 + \ldots + w_nx_n + b))$
  If a point is correctly classified with enough margin, the loss is 0
  If it is misclassified or too close to the boundary, loss > 0
- **SVM Loss Function:**
  $L = \frac{1}{2}(w_1^2 + w_2^2 + \ldots + w_n^2) + C\sum_{i=1}^{m} L_i$
    - **First term:** For maximizing the margin
      (the first term is $\frac{1}{2}||w||^2$, the margin is $\frac{1}{||w||}$
    - **Second term:** Hinge Loss, for reducing classification errors
    - **C** controls the trade-off between margin and classification error

- The objective of the SVM algorithm is to find a decision boundary with the maximum margin, which is the greatest distance from the nearest data point to the decision boundary



- For non-linear separable data, SVM uses **Kernel tricks** to transform the data into a higher dimension where a hyperplane can be used for separation
- Common Kernels: Linear, Polynomial, Radial Basis Function (RBF), Sigmoid. The choice of kernel affects the decision boundary and the performance of the SVM model
- Loss function: Include 2 components:
    - **Hinge Loss**
    - **Regularization Term:** To prevent the model from overfitting, a regularization term is added to the loss function. The most common is **L2 regularization**, which is the squared magnitude of the weight vector w
    - Complete Loss function:

      $$J(w) = \frac{C}{2}||w||^2 + \frac{1}{N}\sum_{i=1}^{N} max(0, 1 - y_i(wx_i - b))$$

    - A large value of C corresponds to assigning a higher penalty to misclassified points, essentially prioritizing the correctness of classification over the width of the margin → Subject to Overfitting (lower bias, higher variance)
    - Conversely, a smaller C value emphasizes a larger margin and allows more misclassifications → Subject to Underfitting (higher bias, lower variance)
- SVM versus Logistic Regression:
    - Let the number of features is m and the number of training samples is n
    - If m >> n → Linear SVM or Logistic Regression
    - If m > n → SVM with Gaussian kernel
    - If m << n → Linear SVM or Logistic Regression

5. KNN (K-Nearest Neighbors)

Predict based on the most similar nearby data points. Similar inputs tend to have similar outputs. KNN predicts the output based on the most similar training examples

- Steps:
    - Choose a value for K (number of neighbors)
    - Compute the distance between the new data point and all training data
    - Find the K nearest neighbors
    - Make prediction based on these neighbors

- Prediction:
    - Classification: Choose the most common class among the neighbors
    - Regression: Take the average value of the neighbors

6. K-Means Clustering

Group similar data points into clusters

- Steps:
    - Choose the number of clusters K
    - Randomly initialize K cluster centers (centroids)
    - Assign each data point to the nearest centroid
    - Recompute each centroid as the mean of its assigned points
    - Repeat steps 3-4 until the centroids become stable

7. Neural Networks

Learn complex patterns through layers of interconnected nodes

- Steps:
    - Initialize the network weights randomly
    - Input data passes through the network to produce a prediction (Forward pass)
    - Compute the loss by comparing prediction with the true label
    - Calculate how each weight affects the loss (backpropagation)
    - Update the weights to reduce the loss (gradient descent)
    - Repeat until the loss becomes small or stable